FESTAVA LIVE

Website Animation Project

Complete Technical Documentation

Tech Stack: HTML | CSS | JavaScript | GSAP 3 | ScrollTrigger | Lenis

Version 1.0 | 2026

Table of Contents

1. Project Overview 3
2. File Structure 3
3. Technology Stack 4
4. HTML Structure 4
5. CSS Architecture 5
6. JavaScript Functions 6
   - 6.1 cursorAnimation() 6
   - 6.2 animateNavbar() 7
   - 6.3 animateHeroHeading() 8
   - 6.4 animateAbout() 9
   - 6.5 animateAboutImage() 10
   - 6.6 animateArtist() 11
   - 6.7 animatePlans() 12
   - 6.8 animateGallery() 13
   - 6.9 animateTextSection() 15
   - 6.10 animateTextSection_SVG() 16
   - 6.11 animateFooter() 17
7. GSAP Core Concepts Used 18
8. ScrollTrigger Reference 19
9. Common Issues & Fixes 20
10. Future Improvements 21

1. Project Overview

Festava Live is a fully animated music festival website built to demonstrate advanced GSAP (GreenSock Animation Platform) techniques. This project was created for a YouTube playlist teaching GSAP animations to students and developers.

The website covers a complete music festival experience — from a video hero section, about page, artist showcase, ticket pricing, image gallery with horizontal scroll, to an animated footer. Every section has a unique animation style built with GSAP and ScrollTrigger.

GOAL	Every section must feel alive. Animations should enhance user experience, not distract from content. Each animation is tied to scroll position or user interaction.

What makes this project special?

- Custom cursor that changes shape on hover over different sections
- Full-screen video background on hero section
- Horizontal gallery scroll pinned with GSAP ScrollTrigger
- 3D tilt effect on artist cards using mousemove
- Wave animation on text characters
- SVG blob that follows cursor in text section
- Footer logo hover reveals a floating image
- Every section has scroll-triggered reveal animations

2. File Structure

```
festava-live/
  |-- index.html           (Main HTML file — all sections)
  |-- style.css            (All CSS styles)
  |-- script.js            (All GSAP animations)
  |-- assets/
       |-- hero-section-video.mp4   (Hero background video)
       |-- about-img.jpg            (About section background)
       |-- about-img-2.jpg          (About section right image)
       |-- artist-1.jpg             (Left big artist)
       |-- artist-2.jpg             (Right artist top)
       |-- artist-3.jpg             (Right artist bottom)
       |-- music-artist-img.avif    (Cursor hover image)
```

All three files work together. HTML defines structure, CSS defines appearance, and JS adds all animations. GSAP is loaded via CDN — no npm or build tools needed.

3. Technology Stack

Technology	Version	Purpose
HTML5	Latest	Structure and semantic markup
CSS3	Latest	Styling, layout, transitions
JavaScript	ES6+	Logic and DOM manipulation
GSAP	3.13.0	All animations (core engine)
ScrollTrigger	3.13.0	Scroll-based animation triggers
Font Awesome	6.5.2	Icons (social, arrows, etc.)

4. HTML Structure

The entire website is one HTML file. It is divided into sections — each section is independently styled and animated. The order of sections matches the page scroll order.

Element	ID / Class	Purpose
div	#cursor .cursor-box	Custom animated cursor element
div	.video-container	Full screen video background
div	.overlay	Dark overlay on top of video
section	.main	Contains navbar + hero
nav	.navbar	Navigation bar with links
div	.hero	Hero content (title, button, status)
section	.about	About section with two columns
section	#artist-section	Artist cards with images
section	#plans-section	Pricing cards
section	.gallery-section	Intro text + horizontal gallery
section	#animation-text-section	Wave text + SVG blob
footer	#footer	Footer with logo, nav, newsletter

NOTE The cursor div (#cursor) is placed at the very top of body — before all sections. This ensures it always renders above everything else due to z-index: 9999.

5. CSS Architecture

The CSS file is organized section by section matching the HTML order. There is no CSS framework — everything is written manually for full control over animations.

Key CSS Techniques Used

CSS Variables

Root variables are not heavily used in this project — colors are mostly hardcoded. In future versions, using CSS variables would make theme changes easier.

position: fixed vs position: absolute

`position: fixed` — elements stay in the same place even when you scroll. Used for: cursor, overlay, video background.

`position: absolute` — elements are positioned relative to their nearest positioned parent. Used for: overlay text on images, about-card, artist-info.

z-index Layers

```
z-index: -2   →  Video background (.video-container)
z-index: -1   →  Dark overlay (.overlay)
z-index:  1   →  Main content (.main)
z-index:  2   →  Artist info overlay (.artist-info)
z-index:  9998 → SVG blob (#blob)
z-index:  9999 → Custom cursor (.cursor-box)
```

will-change: transform

Applied on elements that GSAP will animate using transform. This tells the browser to prepare GPU acceleration for that element in advance, making animations smoother and preventing repaints.

clamp() for Responsive Font Sizes

```
font-size: clamp(3rem, 8vw, 7rem);
           ^min   ^preferred  ^max

// No media queries needed.
// Font scales between 3rem and 7rem based on viewport width.
```

6. JavaScript Functions

All animations are organized into separate functions. Each function handles one section of the website. This makes the code modular — easy to debug, update, or teach one section at a time.

PATTERN	Every function is defined first, then called at the bottom of the file. This separation keeps the code readable. If an animation breaks, you can comment out just that one function call.

6.1 cursorAnimation()

CURSOR

This function makes the orange circle div (#cursor) follow the mouse pointer across the entire page. It creates a custom cursor that replaces the browser default.

How It Works

Every time the mouse moves, a mousemove event fires. We read the current mouse position from clientX and clientY, then use gsap.to() to move the cursor element to that position with a smooth lag effect.

```
document.addEventListener('mousemove', (e) => {
  gsap.to('#cursor', {
    x: e.clientX,   // horizontal position from left edge of viewport
    y: e.clientY,   // vertical position from top edge of viewport
    duration: 1,    // takes 1 second to reach mouse position
    ease: 'power2.out'  // fast start, slow end = natural feel
  });
});
```

Why clientX and not pageX?

clientX gives position relative to the viewport (the visible screen area). pageX gives position relative to the entire page including scrolled content. Since the cursor is position: fixed to the viewport, clientX is the correct choice. Using pageX would make the cursor jump when the page is scrolled.

Why duration: 1 and not 0?

Duration 0 would make the cursor stick directly to the mouse — no lag. The 1 second duration creates a trailing effect where the cursor follows the mouse but slightly behind. This is what gives it the smooth, premium feel. Reduce to 0.3-0.5 for less lag, increase to 1.5 for more.

CSS Requirements

```
.cursor-box {
  position: fixed;              /* stays in place while scrolling */
  top: 0; left: 0;             /* positioned from top-left corner */
  transform: translate(-50%, -50%); /* centers the circle on cursor */
  pointer-events: none;        /* cursor box doesn't block clicks */
  z-index: 9999;               /* always on top of everything */
}
```

KEY INSIGHT transform: translate(-50%, -50%) is critical. Without it, the top-left corner of the cursor div sits on the mouse pointer instead of the center of the circle.

6.2 animateNavbar()

NAVBAR + HERO

This function animates the navbar elements (logo, nav links, button) dropping down from above when the page loads. It also chains the hero section animations after the navbar finishes.

Why a Timeline?

gsap.timeline() chains multiple animations in sequence automatically. Without a timeline, you would need to manually calculate delay values for each element. With a timeline, each animation starts after the previous one finishes — unless you use position parameters.

```
const tl = gsap.timeline();

tl.from('.logo', { y: -200, opacity: 0, duration: 0.5 }, 'start')
  .from('#nav-links li', { y: -200, stagger: 0.2 })   // after logo
  .from('#nav-btn', { y: -200 }, '-=0.5')             // 0.5s before prev ends
  .from('.hero p.gradient', { opacity: 0, y: -20 })   // after button
  .from('.hero h1', { rotateX: 25, rotateZ: -4 });    // after label
```

Understanding the Position Parameter

The third argument in tl.from() controls when an animation starts:

- 'start' — starts at the same time as the 'start' label (simultaneously with logo)
- '-=0.5' — starts 0.5 seconds before the previous animation ends (overlap)
- '+=0.3' — starts 0.3 seconds after the previous animation ends (gap)
- No argument — starts immediately after previous animation ends

stagger: 0.2 on nav links

stagger adds a delay between each element in a group. With stagger: 0.2, the first link drops at 0s, second at 0.2s, third at 0.4s, and so on. This creates a cascading waterfall effect instead of all links dropping together.

rotateX and rotateZ on Hero Heading

```
gsap.from('.hero h1', {
  rotateX: 25,               // tilts back (top away, bottom toward viewer)
  rotateZ: -4,               // slight counter-clockwise tilt
  transformPerspective: 1000, // camera distance — lower = more dramatic 3D
  transformOrigin: 'left center', // rotation pivots from left edge
});
```

WHY BOTH Using rotateX alone looks mechanical. Adding a slight rotateZ (-4 degrees) makes it feel like the text is naturally falling into place — like a page turning.

6.3 animateHeroHeading()

HERO TEXT SPLIT

This function splits the hero heading text (Night Live 2026) into individual character spans, then animates the left half from the left and the right half from the right, meeting in the center.

Step 1 — Split text into characters

```
const headingText = headingElement.textContent; // 'Night Live 2026'
const textArray = headingText.split('');         // ['N','i','g','h','t',' ','L'...]
const halfLength = textArray.length / 2;         // 7.5 (middle point)
```

Step 2 — Wrap each character in a span

```
textArray.forEach((char, index) => {
  if (char === ' ') {
    // Space needs special handling — plain span collapses
    newText += `<span style='display:inline-block; width:0.3em'> </span>`;
  } else if (index < halfLength) {
    newText += `<span class='left-span'>${char}</span>`;
  } else {
    newText += `<span class='right-span'>${char}</span>`;
  }
});
headingElement.innerHTML = newText;
```

Step 3 — Animate left and right halves

```
gsap.from('#heading .left-span', {
  y: 100,       // comes from 100px below
  opacity: 0,
```

```
  duration: 0.6,
  stagger: 0.1  // left to right wave
});

gsap.from('#heading .right-span', {
  y: 100,
  opacity: 0,
  duration: 0.6,
  stagger: -0.1  // negative = right to left wave (meets center)
});
```

KEY display: inline-block on each span is mandatory in CSS. GSAP cannot animate transform (x, y, rotation) on inline elements. inline-block makes each character an independent block that can be moved.

6.4 animateAbout()

ABOUT SECTION

This function animates all elements of the about section as the user scrolls into view. Left content slides from the left, right image slides from the right, and the floating card bounces in.

ScrollTrigger Basics

```
scrollTrigger: {
  trigger: '.about .left',  // which element to watch
  start: 'top 80%',         // when element's top reaches 80% down viewport
  end: 'top 30%',           // when element's top reaches 30% down viewport
  scrub: 0.5,               // animation tied to scroll (0.5 = smooth lag)
}
```

Understanding start and end

The format is: 'element-position viewport-position'

- 'top 80%' — element's top edge at 80% down the viewport (near bottom of screen)
- 'top 30%' — element's top edge at 30% down the viewport (upper area of screen)
- So animation plays while you scroll from when element enters screen to when it's well inside

scrub: true vs scrub: 1

- scrub: true — animation is directly tied to scroll position, 1:1 mapping, no smoothing
- scrub: 0.5 — animation follows scroll but with 0.5 second lag (smoother feel)
- scrub: 1 — animation follows scroll with 1 second lag (even smoother)
- No scrub — animation plays once when triggered, not tied to scroll position

Background Parallax Effect

```
gsap.to('.bg-img-box img', {
  yPercent: 20,     // image moves 20% of its height upward during scroll
  ease: 'none',     // linear — no easing for parallax (constant speed)
  scrollTrigger: {
    trigger: '.about',
    start: 'top bottom',   // when section enters from bottom
    end: 'bottom top',     // when section exits from top
    scrub: true,           // perfectly tied to scroll
  }
});
```

PARALLAX LOGIC	The container has overflow: hidden. The image inside is larger than the container. As you scroll down, the image moves up — creating the illusion of depth. The faster the image moves compared to the page, the stronger the parallax effect.

6.5 animateAboutImage()

CURSOR MORPH

When the user hovers over the about section's right image, the cursor transforms from a small orange circle into a large rectangle that shows an image inside it. This is called cursor morphing.

How the cursor morphs

```
// On mouseenter — cursor expands and shows image
aboutImage.addEventListener('mouseenter', () => {
  gsap.to('#cursor', {
    width: 180,          // grows from 40px to 180px
    height: 220,         // grows from 40px to 220px
    borderRadius: '12px', // circle becomes rectangle
    duration: 0.5,
    ease: 'power3.out',
  });
  cursor.classList.add('show-img');  // fades in the inner image
});

// On mouseleave — cursor shrinks back to circle
aboutImage.addEventListener('mouseleave', () => {
  gsap.to('#cursor', {
    width: 40,
    height: 40,
    borderRadius: '50%',  // rectangle becomes circle again
    duration: 0.4,
  });
  cursor.classList.remove('show-img');
});
```

The show-img CSS class

```
/* Default — image hidden */
.cursor-inner-img { opacity: 0; transition: opacity 0.3s ease; }

/* When show-img class added — image fades in */
.cursor-box.show-img .cursor-inner-img { opacity: 1; }

/* Note: .cursor-box.show-img (no space) means show-img is ON cursor-box itself */
/* .cursor-box .show-img (with space) would look INSIDE cursor-box — wrong! */
```

COMMON MISTAKE .cursor-box .show-img (space between) means 'find show-img inside cursor-box'. .cursor-box.show-img (no space) means 'cursor-box that also has show-img class'. The second is correct here.

6.6 animateArtist()

ARTIST CARDS

This function handles three types of animation for the artist section: cursor hiding (replaced with system cursor), scroll-triggered card entrance, and 3D tilt effect on mousemove.

Part 1 — Hide custom cursor in artist section

```
artistSection.addEventListener('mouseenter', () => {
  cursor.style.display = 'none';  // hide GSAP cursor
});
artistSection.addEventListener('mouseleave', () => {
  cursor.style.display = 'flex';  // show GSAP cursor again
});
```

The artist cards have their own hover animations. Showing the custom cursor on top would look cluttered. So we hide it when entering the artist section.

Part 2 — 3D Tilt Effect on Mousemove

```
card.addEventListener('mousemove', (e) => {
  const rect = card.getBoundingClientRect();

  // Mouse position as 0-1 value within the card
  const x = (e.clientX - rect.left) / rect.width;
  const y = (e.clientY - rect.top) / rect.height;

  // Convert to -0.5 to +0.5 range (0 at center)
  const rotateY = (x - 0.5) * 12;  // left-right tilt, max 12 degrees
  const rotateX = -(y - 0.5) * 12; // up-down tilt, max 12 degrees

  gsap.to(card, {
    rotateX, rotateY,
    scale: 1.03,             // slight zoom
    transformPerspective: 900,
```

```
    duration: 0.4,
    ease: 'power2.out'
  });

  // Image moves OPPOSITE direction — depth illusion
  gsap.to(card.querySelector('img'), {
    x: (x - 0.5) * -15,
    y: (y - 0.5) * -15,
    duration: 0.4
  });
});
```

Why getBoundingClientRect()?

getBoundingClientRect() returns the exact position and size of an element relative to the viewport. We need this to calculate where the mouse is inside the card (0 to 1 value). Without it, we can't know if the mouse is at the left edge or right edge of the card.

Why does the image move opposite to the card?

The card tilts right, but the image inside moves left. This creates a parallax effect inside the card — as if there's depth between the card surface and the image. This is what makes it feel truly 3D rather than just a flat tilt.

elastic.out on mouseleave

```
card.addEventListener('mouseleave', () => {
  gsap.to(card, {
    rotateX: 0, rotateY: 0, scale: 1,
    ease: 'elastic.out(1, 0.6)',  // spring back effect
    duration: 0.7
  });
});

// elastic.out(amplitude, period)
// amplitude: 1 = normal spring strength
// period: 0.6 = how quickly spring settles (lower = faster)
```

6.7 animatePlans()

PRICING CARDS

The two pricing cards slide in from opposite sides when the section scrolls into view. Left card comes from the left, right card comes from the right, with a slight 3D rotation.

```
gsap.from('#left-plan', {
  x: -300,              // starts 300px to the left
  opacity: 0,
  rotateX: 45,          // tilted backward (top away from viewer)
```

```
  transformPerspective: 900,
  ease: 'power3.out',
  scrollTrigger: {
    trigger: '#plans-section',
    start: 'top 80%',
    toggleActions: 'play none none none',
  }
});
```

toggleActions vs scrub

toggleActions: 'play none none none' means:

- onEnter: play — animation plays when section enters viewport
- onLeave: none — nothing when section leaves
- onEnterBack: none — nothing when scrolling back up
- onLeaveBack: none — nothing when scrolling back down

This is used instead of scrub when you want a one-time animation that plays and stays — not tied to scroll position.

6.8 animateGallery()

HORIZONTAL SCROLL

This is the most complex function in the project. It handles two things: the intro text animations, and the pinned horizontal scroll gallery.

Part 1 — Intro Text Animation

A standard scrub-based timeline that reveals the intro heading and subtitle as you scroll into the gallery section.

Part 2 — Horizontal Scroll (The Core Logic)

```
const wrapper = document.querySelector('#gallery-img-wrapper');
const totalScrollWidth = wrapper.scrollWidth - window.innerWidth;
// scrollWidth = total width of all images side by side
// window.innerWidth = visible screen width
// Difference = how far we need to scroll horizontally

const galleryTween = gsap.to(wrapper, {
  x: -totalScrollWidth,   // move left by that much
  ease: 'none',           // linear (no easing = consistent scroll feel)
  scrollTrigger: {
    trigger: '.gallery-wrapper',
    start: 'top top',     // pin when wrapper hits top of viewport
    end: () => `+=${totalScrollWidth + 200}`, // scroll this much before unpinning
    pin: true,            // freeze section while horizontal scroll happens
    scrub: 1,             // smooth 1 second lag
```

```
    anticipatePin: 1,     // prevents jump when pinning starts
  }
});
```

Why use an arrow function for end?

```
// WRONG — calculated once on page load
end: `+=${totalScrollWidth}`,

// CORRECT — recalculated on resize
end: () => `+=${totalScrollWidth}`,

// Arrow function means ScrollTrigger calls it fresh each time
// it needs the value — including on window resize.
// Static string would break on mobile or window resize.
```

containerAnimation — Individual Image Triggers

```
gsap.from(img, {
  opacity: 0, scale: 0.8,
  scrollTrigger: {
    trigger: img,
    containerAnimation: galleryTween,  // KEY: tells ScrollTrigger to
    start: 'left 85%',                 // watch horizontal position,
    // not vertical page scroll
  }
});

// Without containerAnimation, ScrollTrigger uses vertical scroll.
// With it, ScrollTrigger watches the horizontal galleryTween progress.
// 'left 85%' = when image's left edge reaches 85% from left of viewport
```

PIN CONCEPT	pin: true freezes the .gallery-wrapper element in place. The page keeps scrolling but the section stays fixed. GSAP uses that scroll progress to drive the horizontal x movement instead. So vertical scroll = horizontal movement.

CSS Requirements for Horizontal Scroll

```
.gallery-wrapper  { overflow: hidden; }     /* clip images */
.gallery-img-wrapper {
  display: flex;
  flex-wrap: nowrap;       /* MUST — prevents wrapping to new row */
  overflow: visible;       /* NOT overflow-x: auto — GSAP handles scroll */
  will-change: transform;  /* GPU acceleration hint */
}
.gallery-img-wrapper img {
  flex-shrink: 0;          /* MUST — prevents images from shrinking */
  height: 80vh;
  width: auto;
}
```

6.9 animateTextSection()

WAVE TEXT

This function creates a continuous wave animation on the large heading text 'LANGUAGE OF THE SOUL'. Each character bobs up and down with a slight delay from the previous, creating a fluid wave effect.

```
// Each character becomes a span
textArray.forEach((char, index) => {
  newText += `<span>${char}</span>`;
});

// Animate all spans with stagger
gsap.to('#anim-big span', {
  y: -50,           // moves up 50px
  duration: 0.6,
  ease: 'sine.inOut',
  stagger: {
    each: 0.05,     // 0.05s delay between each character
    repeat: -1,     // loops forever
    yoyo: true,     // reverses back down after going up
  },
  repeat: -1,
});
```

Why sine.inOut for wave?

sine.inOut creates a smooth S-curve acceleration. The character starts slow, speeds up in the middle, and slows down again at the peak. This matches the natural motion of a wave — making it feel organic rather than mechanical.

How stagger creates the wave

Without stagger, all characters move up at the same time — looks like a block jumping. With stagger: 0.05, character 1 starts at 0s, character 2 at 0.05s, character 3 at 0.1s, etc. Each character is in a slightly different part of its animation at any given moment — this offset is what creates the wave visual.

6.10 animateTextSection_SVG()

SVG BLOB

This function makes an SVG shape (a rounded star/blob shape) follow the cursor when the mouse moves inside the animation text section. It also hides the main cursor while inside this section.

```
// Hide main cursor when entering section
section.addEventListener('mouseenter', () => {
  cursor.style.display = 'none';
```

```
});

// SVG blob follows mouse
section.addEventListener('mousemove', (e) => {
  gsap.to('#blob', {
    x: e.clientX - 60,   // offset so blob center aligns with cursor
    y: e.clientY - 60,   // (blob is 120px, so -60 centers it)
    duration: 0.3,
    ease: 'power2.out'
  });
});

// Show cursor again when leaving
section.addEventListener('mouseleave', () => {
  cursor.style.display = 'flex';
});
```

CSS for the SVG blob

```
#blob {
  position: fixed;          /* stays relative to viewport */
  top: 0; left: 0;          /* GSAP x/y moves it from here */
  width: 120px; height: 120px;
  pointer-events: none;     /* doesn't block mouse events */
  z-index: 9998;            /* below cursor (9999) but above content */
}
```

WHY - 60 OFFSET	GSAP sets x and y from the element's top-left corner. The blob is 120px wide. To center it on the cursor, we subtract half the width (60px) from both x and y. This puts the center of the blob on the cursor position.

6.11 animateFooter()

FOOTER

This combined function handles: cursor hiding in footer, logo hover image reveal, and all scroll-triggered entrance animations for footer elements.

Logo Hover Image Effect

```
logo.addEventListener('mouseenter', () => {
  gsap.to(image, { opacity: 1, scale: 1, duration: 0.4 });
});

logo.addEventListener('mousemove', (e) => {
  gsap.to(image, {
    // Position relative to footer, not viewport
    x: e.clientX - footer.getBoundingClientRect().left - 150,
    y: e.clientY - footer.getBoundingClientRect().top - 190,
    duration: 0.5,
  });
```

```
});

logo.addEventListener('mouseleave', () => {
  gsap.to(image, { opacity: 0, scale: 0.8, duration: 0.3 });
});
```

Why subtract getBoundingClientRect()?

The image (#footer-logo-img) is position: absolute inside the footer. Its x and y are relative to the footer's top-left corner, NOT the viewport. But clientX/clientY are relative to the viewport. So we subtract the footer's position from the viewport to convert viewport coordinates to footer-relative coordinates.

Footer Scroll Animations Summary

- footer-logo — slides in from left (x: -100)
- footer-tagline — slides in from left with delay
- footer-divider — grows from top (scaleY: 0 to 1)
- footer-nav-items — stagger fade in from left
- footer-newsletter — slides in from right (x: 80)
- social icons — scale up with back.out bounce
- footer-ghost-text 'LIVE' — slides in from right (x: 200)

7. GSAP Core Concepts Used

Method	Syntax	What it does
gsap.from()	`gsap.from(el, { x: -100 })`	Animates FROM values TO current state
gsap.to()	`gsap.to(el, { x: 100 })`	Animates FROM current state TO values
gsap.timeline()	`const tl = gsap.timeline()`	Chain animations in sequence
stagger	`stagger: 0.1`	Delay between multiple elements
yoyo	`yoyo: true`	Reverses animation back to start
repeat	`repeat: -1`	Loop animation (-1 = infinite)
scrub	`scrub: 1`	Tie animation to scroll position
pin	`pin: true`	Freeze element during scroll
ease	`ease: 'power3.out'`	Control acceleration curve
transformPerspective	`transformPerspective: 900`	3D depth for rotations

Easing Functions Quick Reference

```
power1.out   →  Gentle deceleration (subtle)
power2.out   →  Medium deceleration (common use)
power3.out   →  Strong deceleration (good for reveals)
power4.out   →  Very strong deceleration (dramatic)
back.out(x)  →  Overshoots then settles (x = intensity)
elastic.out  →  Spring/bounce effect
sine.inOut   →  Smooth S-curve (good for waves/loops)
none/linear  →  Constant speed (good for scroll-tied animations)
```

8. ScrollTrigger Reference

start and end Format

Format: 'trigger-position scroller-position'

```
start: 'top 80%'
       ^           ^
       |           |__ viewport position (80% down from top of screen)
       |_____________ element position (top edge of element)

Common values for element position: top, center, bottom
Common values for viewport position: top, center, bottom, 80%, 50%

Examples:
  'top bottom'   = trigger when element top hits bottom of screen (first visible)
  'top center'   = trigger when element top reaches middle of screen
  'top top'      = trigger when element top reaches top of screen (pinning)
  'bottom top'   = trigger when element bottom reaches top (element fully scrolled
past)
```

toggleActions

```
toggleActions: 'onEnter  onLeave  onEnterBack  onLeaveBack'

Values: play, pause, resume, reset, restart, complete, reverse, none

Most common:
  'play none none none'     = play once when entering, never again
  'play reverse none none'  = play forward entering, reverse when leaving
  'restart none none none'  = restart every time element enters viewport
```

markers: true (Debug Tool)

```
scrollTrigger: {
  markers: true,  // Shows green (start) and red (end) lines on page
}

// IMPORTANT: Remove markers: true before final deployment
// They are only for debugging to see exactly where triggers fire
```

9. Common Issues & Fixes

Problem	Fix
GSAP target null not found	Element doesn't exist in DOM. Check ID/class spelling. Make sure JS runs after HTML loads.
Animation plays on page load, not scroll	ScrollTrigger not set up. Add scrollTrigger: {} block with trigger and start.
Horizontal scroll doesn't work	Check flex-wrap: nowrap and overflow: visible on track. Remove overflow-x: auto.
3D rotation looks flat	Add transformPerspective: 900. Without it, 3D rotations have no depth.
Cursor not centered on mouse	Add transform: translate(-50%, -50%) in CSS to the cursor element.
span characters not animating	Add display: inline-block to span in CSS. GSAP can't transform inline elements.
Space disappears between words	Use or width: 0.3em on space spans. Plain spaces in spans collapse.
markers showing in final site	Remove markers: true from all scrollTrigger objects before deployment.
Image cursor shows wrong	.cursor-box .show-img is wrong selector. Use .cursor-box.show-img (no space).
Gallery jumps when pinning	Add anticipatePin: 1 to the scrollTrigger config.

10. Future Improvements

These are features that can be added to make the project even more advanced:

Animation Improvements

- Use GSAP SplitText plugin for proper text splitting (handles multi-line, RTL, special chars)
- Add Lenis smooth scroll for butter-smooth page scrolling
- Add page loader animation that plays before content appears
- Add cursor trail effect — multiple dots following the cursor
- Add magnetic button effect — buttons attract cursor when nearby

Section Improvements

- Event schedule section with GSAP-animated table cells
- Countdown timer to the festival date with animated digits
- Music player bar at bottom that plays festival preview audio
- Artists section — clicking an artist card flips it to show bio
- Gallery — add lightbox when clicking an image

Performance

- Lazy load gallery images with IntersectionObserver
- Use will-change: transform only on elements actively animating
- Add prefers-reduced-motion media query to disable animations for accessibility
- Compress video with lower bitrate for faster load times

Code Quality

- Extract all hardcoded colors to CSS variables
- Add GSDevTools for animation debugging in development
- Split JS into multiple files — one per section — and bundle with a build tool
- Add TypeScript for type safety on complex animation configs

FESTAVA LIVE — Technical Documentation

Built with GSAP 3 + ScrollTrigger | 2026